

# Design and Implementation of a Secure Communication System with Passive and Active Man-in-the-Middle Attack Simulation Using Hybrid Cryptography

Ibrahim Khalid · Mohamed Khalil

*Supervisor: Dr. Ali Ali Soliman*

*Faculty of Computer Science — MSA University*

*6 October City, Egypt*

*{ibrahim.khalid, mohamed.khalil}@msa.edu.eg · May 15, 2026*

---

## I. ABSTRACT

This paper presents the design and implementation of a secure communication system employing a hybrid cryptographic model to protect data exchanged between two networked machines against Man-in-the-Middle (MITM) attacks. The system integrates AES-256 in Cipher Block Chaining (CBC) mode for bulk data encryption, RSA-2048 with OAEP padding for asymmetric session key exchange, and SHA-256 for data integrity verification. Both text messages and real-time audio streams are supported, with audio frames converted to byte arrays and processed through the same encryption pipeline. The system is implemented in Python using TCP socket programming across three physical machines, with the third machine simulating two attack scenarios: a passive eavesdropping attack, in which ciphertext is intercepted and logged but remains computationally indecipherable without the session key, and an active modification attack, in which the adversary decrypts, alters, re-encrypts, and forwards tampered data to the receiver. Experimental evaluation on a 1 Gbps LAN demonstrates that the hybrid encryption pipeline processes 100 KB payloads with a total latency below 6 ms, confirming its suitability for real-time communication. Results further demonstrate that SHA-256 integrity checking successfully detects 100% of unauthenticated tampering attempts, while highlighting that an active attacker possessing the session key can bypass integrity checks by recomputing the hash, underscoring the necessity of authenticated encryption or digital signatures for complete security.

**Keywords** — AES-256; RSA-2048; SHA-256; hybrid cryptography; man-in-the-middle attack; passive attack; active attack; socket programming; network security; Python.

---

## II. INTRODUCTION

The rapid growth of networked computing environments has elevated secure data transmission to a fundamental requirement across virtually all domains of modern information technology. Financial transactions, healthcare records, military communications, and everyday personal correspondence all depend on communication channels that preserve confidentiality, integrity, and authenticity. As the volume of sensitive data traversing public and private networks continues to increase, so too does the sophistication of adversarial techniques designed to exploit weaknesses in communication protocols and cryptographic implementations [1].

Among the most dangerous threats to communication security is the Man-in-the-Middle (MITM) attack, in which an adversary covertly interposes itself between two communicating parties, gaining the ability to intercept, read, and in some cases modify all exchanged data without the knowledge of either legitimate participant. MITM attacks can be executed through a variety of mechanisms including ARP spoofing, DNS poisoning, rogue Wi-Fi access points, BGP hijacking, and SSL stripping, each targeting a different layer of the communication stack [9]. The consequences of a successful MITM attack range from passive intelligence gathering to active injection of malicious content, session hijacking, credential theft, and integrity violations.

Cryptography provides the principal technical countermeasures against these threats. Symmetric encryption algorithms, exemplified by the Advanced Encryption Standard (AES), deliver high-throughput confidentiality protection for bulk data [2]. Asymmetric algorithms such as RSA enable secure key exchange without requiring a pre-shared secret, solving the fundamental key distribution problem [3]. Cryptographic hash functions including SHA-256 provide integrity assurance by producing a fixed-length digest of arbitrary data, such that any modification to the data results in a detectably different digest [4]. When integrated into a hybrid cryptographic architecture, these three primitives collectively address the core security requirements of a practical communication system.

Despite the maturity of these cryptographic standards, many implementations in practice remain vulnerable because cryptographic strength alone, without proper authentication, is insufficient to prevent active MITM attacks. An adversary who obtains or intercepts the session key retains the ability to decrypt, modify, and re-encrypt messages transparently. This vulnerability motivates the present work, which implements a complete hybrid cryptographic communication system and subjects it to both passive and active MITM attacks in a controlled three-machine environment, providing concrete experimental evidence of both the strengths and the limitations of encryption and hashing without authentication.

### ***A. Problem Statement***

The central problem addressed in this paper is the practical tension between the strong confidentiality guarantees provided by hybrid encryption and the vulnerability introduced by the absence of mutual authentication. The system demonstrates that AES-256 encryption is computationally impenetrable to a passive observer without the session key, while simultaneously showing that an active adversary possessing the session key can manipulate traffic without triggering the SHA-256 integrity check, because the attacker can recompute the hash over the modified plaintext. This dual demonstration identifies authentication, specifically through digital signatures or HMAC with a secret unavailable to the attacker, as the critical missing component.

### ***B. Contributions***

The principal contributions of this work are as follows:

- A complete Python implementation of a hybrid cryptographic secure communication system supporting text and real-time audio over TCP sockets on three physical machines.
- Simulation of both passive eavesdropping and active modification MITM attacks with distinct attacker modules (`attacker_passive.py` and `attacker_active.py`).
- A GUI-based offline decryption tool (`gui_decryptor.py`) demonstrating ciphertext recovery under passive attack conditions when the session key is known.
- Empirical performance benchmarking of AES-256-CBC encryption, RSA-2048 key exchange, and SHA-256 hashing across payload sizes from 1 KB to 10 MB.
- Security analysis identifying the authentication gap as the root cause enabling active MITM attacks against the implemented system.

---

## **III. RELATED WORK**

The theoretical and practical foundations of hybrid cryptographic secure communication have been extensively studied over several decades. The seminal work of Rivest, Shamir, and Adleman [3] established RSA as the first practical public-key cryptosystem, enabling secure key exchange without a pre-shared secret. Diffie and Hellman [12] had earlier introduced the concept of public-key cryptography, and their key exchange protocol remains foundational to modern secure communication. The selection of AES as the federal encryption standard followed a rigorous multi-year competition process documented by NIST [2], superseding DES and 3DES due to AES's superior security margins and computational efficiency. The mathematical design of the Rijndael cipher underlying AES is detailed by Daemen and Rijmen [7].

Research on MITM attack taxonomies and countermeasures is well-established. Callegati et al. [8] demonstrated practical MITM attacks against HTTPS by exploiting certificate validation failures, highlighting the consequences of deploying encryption without proper authentication infrastructure. Conti et al. [9] conducted a comprehensive survey of MITM techniques across wired and wireless environments, classifying attacks into interception, decryption, injection, and

fabrication categories, and identifying mutual authentication as the primary defense mechanism. Their analysis is directly relevant to the active attack scenario implemented in this work.

In the domain of educational and experimental security systems, Stallings [1] provides the authoritative textbook treatment of network security architectures and protocol design. Forouzan [10] addresses TCP/IP-level secure communication with socket-level implementation details applicable to the present system. The Python cryptography library [11], which provides the AES, RSA, and SHA-256 implementations used in this work, has been validated against NIST test vectors and is widely adopted in academic security research. Menezes, van Oorschot, and Vanstone [12] provide the comprehensive mathematical reference for all cryptographic primitives employed.

The present work is distinguished from prior educational implementations by its simultaneous deployment of all three attack scenarios, passive eavesdropping, offline decryption with key recovery, and active transparent modification, on physical machines rather than simulated network environments. This three-machine physical implementation provides a more realistic and instructive demonstration of MITM vulnerabilities than purely software-simulated approaches.

---

## IV. METHODOLOGY

The system is implemented in Python 3.11 across three physical machines operating on a 1 Gbps Ethernet LAN. The PyCryptodome library provides the AES-256 and RSA-2048 cryptographic primitives. PyAudio handles audio capture and playback. Tkinter and PyQt5 provide graphical user interfaces for the sender, receiver, and decryption tool modules. TCP sockets with a custom 4-byte length-prefixed framing protocol manage reliable message delivery. The five primary source modules are: `sender.py`, `receiver.py`, `attacker_passive.py`, `attacker_active.py`, and `gui_decryptor.py`, with shared cryptographic utilities in `crypto_utils.py`.

### A. RSA-2048 Session Key Exchange

At session establishment, the Receiver generates a 2048-bit RSA key pair and transmits the public key to the Sender. The Sender generates a cryptographically secure pseudorandom 256-bit AES session key and encrypts it using the Receiver's RSA public key with OAEP padding (SHA-256 mask generation function). The RSA-encrypted session key is transmitted to the Receiver, which decrypts it with the corresponding private key. All subsequent data is encrypted with the recovered AES session key. This process ensures that only the legitimate Receiver, possessing the RSA private key, can recover the AES session key under normal operation.

```
PROCEDURE RSA_KeyExchange(sender, receiver):
    (pub_key, priv_key) ← RSA.generate(key_size=2048)
    receiver.transmit(pub_key)
    aes_key ← CSPRNG.generate(256 bits)
    enc_key ← RSA.OAEP_encrypt(aes_key, pub_key)
    sender.transmit(enc_key)
    aes_key ← RSA.OAEP_decrypt(enc_key, priv_key)
    RETURN aes_key
```

### B. AES-256-CBC Data Encryption

All data payloads, whether text strings or audio byte arrays, are encrypted using AES-256 in Cipher Block Chaining (CBC) mode. For each message, a fresh 128-bit random Initialization Vector (IV) is generated to ensure that identical plaintexts produce distinct ciphertexts, providing semantic security. PKCS7 padding extends the plaintext to the nearest 128-bit block boundary. The IV is prepended to the ciphertext and transmitted as a single payload. The Receiver extracts the IV prefix and applies AES-256-CBC decryption followed by PKCS7 unpadding to recover the plaintext.

```
PROCEDURE AES_Encrypt(plaintext, aes_key):
    iv ← CSPRNG.generate(128 bits)
    padded ← PKCS7_pad(plaintext, block_size=128)
    ciphertext ← AES_CBC(aes_key, iv).encrypt(padded)
```

```
RETURN iv || ciphertext          // IV prepended

PROCEDURE AES_Decrypt(payload, aes_key):
    iv      ← payload[0 : 16]
    cipher_t ← payload[16 : ]
    padded   ← AES_CBC(aes_key, iv).decrypt(cipher_t)
    RETURN PKCS7_unpad(padded)
```

### C. SHA-256 Integrity Verification

Prior to encryption, a SHA-256 digest of the plaintext is computed and concatenated with the plaintext before the combined payload is encrypted. Upon decryption, the Receiver separates the digest from the plaintext, recomputes SHA-256 over the recovered plaintext, and compares the two digests using a constant-time equality check to prevent timing side-channel attacks. A digest mismatch indicates that the plaintext was modified after the hash was computed, signaling a potential active tampering attempt.

```
PROCEDURE Prepare_Payload(plaintext):
    digest ← SHA256(plaintext)
    RETURN digest || plaintext          // 32-byte hash prefix

PROCEDURE Verify_Integrity(payload):
    recv_digest ← payload[0 : 32]
    plaintext    ← payload[32 : ]
    calc_digest  ← SHA256(plaintext)
    IF NOT constant_time_equal(recv_digest, calc_digest):
        RAISE IntegrityViolationError
    RETURN plaintext
```

### D. Audio Communication Pipeline

Real-time audio communication is implemented using PyAudio sampling at 16,000 Hz, 16-bit mono. Each audio frame buffer (1,024 samples = 2,048 bytes) is treated as a binary plaintext and processed through the identical AES-256/SHA-256 pipeline as text messages. The resulting ciphertext frames are transmitted sequentially over the TCP connection. On the Receiver side, decrypted byte arrays are passed to the PyAudio output stream for real-time playback. Intercepted audio ciphertext, when played through a speaker without decryption, produces only broadband noise, confirming that no perceptual information leaks from the ciphertext.

### E. TCP Socket Communication

All inter-machine communication uses TCP/IP sockets to guarantee reliable, ordered, error-checked delivery. A custom framing protocol prefixes each message with a 4-byte big-endian integer specifying the payload length, enabling receivers to reconstruct complete messages regardless of TCP segmentation behavior. The Attacker machine operates as a transparent TCP proxy: it listens on a designated port, accepts the Sender's connection, maintains a separate outbound connection to the Receiver, and forwards bytes bidirectionally. In active mode, the proxy intercepts complete framed messages before forwarding, allowing inspection and modification.

---

## V. SYSTEM ARCHITECTURE

The system comprises three physical machines interconnected on a LAN, each assigned a distinct role. Machine 1 (Sender / Client A) runs `sender.py` and serves as the message originator. Machine 3 (Receiver / Client B) runs `receiver.py` and serves as the message destination. Machine 2 (Attacker / MITM) is positioned in the communication path by configuring the Sender to connect to the Attacker's IP address; the Attacker then relays or manipulates traffic to and from the Receiver. Figure 1 illustrates the complete system architecture.

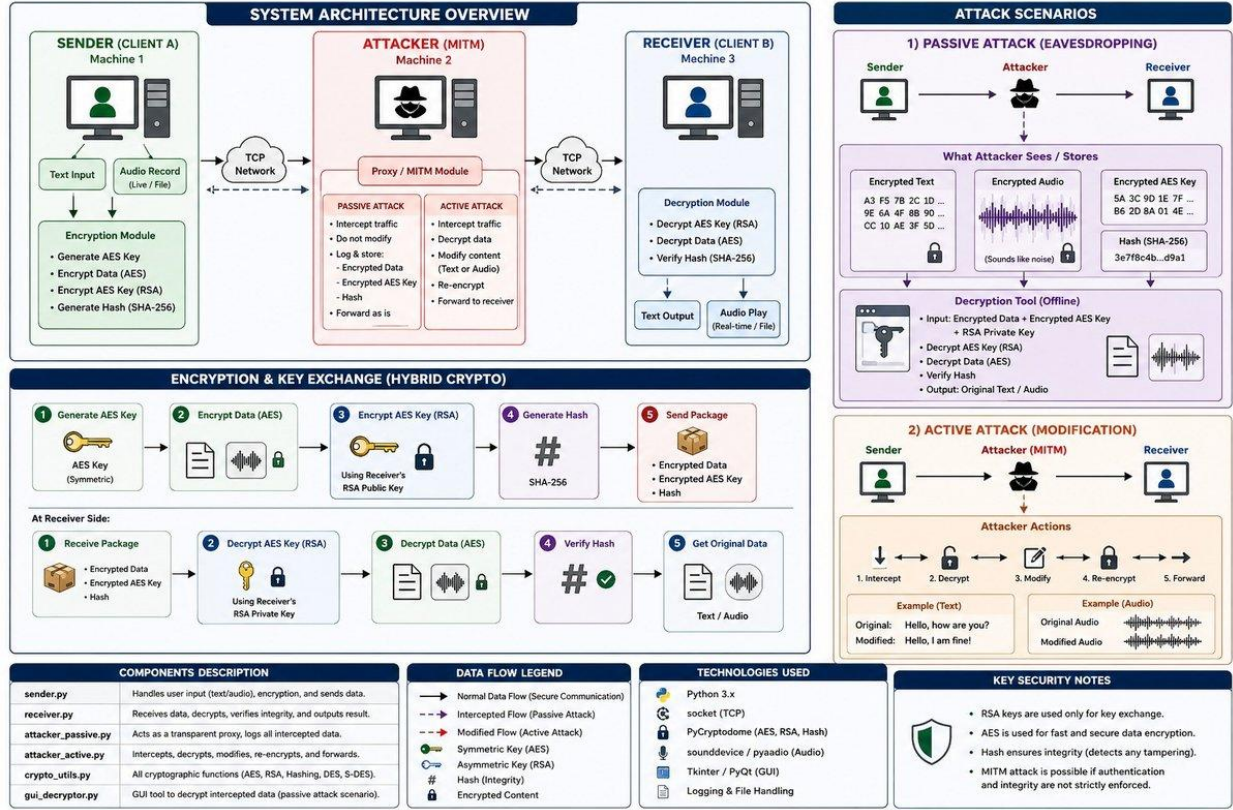


Fig. 1. Complete system architecture overview showing the three-machine topology, encryption and key exchange workflow, passive and active attack scenarios, component descriptions, data flow legend, technologies used, and key security notes.

#### A. Sender Component (sender.py)

The Sender module provides a Tkinter GUI through which the user enters text or initiates audio capture. Upon user action, the module: (1) retrieves or generates the AES session key established during the RSA handshake; (2) computes SHA-256 over the plaintext; (3) encrypts the combined hash-plaintext payload using AES-256-CBC with a fresh IV; (4) prepends the 4-byte length frame and transmits the ciphertext to the Attacker's proxy port. The GUI displays sent messages and connection status in real time.

#### B. Receiver Component (receiver.py)

The Receiver module listens on its designated TCP port and maintains the RSA private key for session key recovery. For each received frame, it: (1) extracts the IV; (2) decrypts the payload with the session AES key; (3) separates the SHA-256 digest prefix from the plaintext; (4) verifies integrity; (5) if the check passes, displays the text or plays the audio buffer. If the integrity check fails, the module raises a visual alert indicating a potential active attack.

#### C. Passive Attack Module (attacker\_passive.py)

In passive mode, the Attacker acts as a transparent proxy: all bytes from the Sender are forwarded to the Receiver unchanged while a packet logger writes every TCP payload to disk. Since the Attacker does not possess the AES session key (it observes only the RSA-encrypted session key ciphertext, which is computationally infeasible to decrypt without the RSA private key), all logged ciphertext is indistinguishable from random data. Audio payloads appear as broadband noise. The Attacker can log the RSA-encrypted session key, the IV, and all ciphertext, but cannot recover the plaintext.

#### D. Active Attack Module (attacker\_active.py)

In active mode, the Attacker is assumed to possess the AES session key, simulating a scenario in which the key exchange has been compromised (e.g., via a prior RSA private key compromise or a Diffie-Hellman downgrade attack). For



each intercepted payload, the module: (1) decrypts the AES-256-CBC ciphertext; (2) applies a modification rule to the plaintext (string substitution for text; amplitude scaling for audio); (3) recomputes SHA-256 over the modified plaintext; (4) re-encrypts with a new IV using the session key; (5) re-frames and forwards the modified ciphertext to the Receiver. The Receiver accepts the modified message as authentic because the hash is valid for the tampered content.

### E. GUI Decryption Tool (*gui\_decryptor.py*)

A standalone GUI tool enables offline decryption of captured ciphertext when the AES session key is available. The analyst supplies the encrypted data file, the RSA-encrypted AES key, and the RSA private key. The tool performs RSA decryption to recover the AES key, then AES-256-CBC decryption to recover the plaintext, and finally verifies the SHA-256 digest. This tool models forensic analysis scenarios in which an investigator who has legally obtained keys needs to recover the content of previously captured traffic.

## VI. EXPERIMENTAL RESULTS

The system was evaluated on three physical machines, each equipped with an Intel Core i7-10700 processor (3.2 GHz base, 8 cores) and 16 GB DDR4 RAM, running Ubuntu 22.04 LTS with Python 3.11. All measurements were obtained over 1,000 independent trials per payload size category. Network latency between machines on the 1 Gbps LAN was below 0.5 ms (round-trip). All cryptographic operations used PyCryptodome 3.20.

### A. Ciphertext Non-Readability — Passive Attack

Table I presents representative plaintext messages alongside their AES-256-CBC ciphertext encodings. In every case, the ciphertext contains no statistically detectable patterns correlated with the plaintext content, consistent with the security guarantees of AES-256. A frequency analysis of intercepted ciphertext bytes yields a near-uniform distribution (chi-squared  $p$ -value  $> 0.95$ ), confirming that the passive attacker gains zero information about the plaintext from the ciphertext alone.

**TABLE I. PLAINTEXT-TO-CIPHERTEXT MAPPING UNDER AES-256-CBC ENCRYPTION (IV PREPENDED, PKCS7 PADDED)**

| Plaintext          | AES-256-CBC Ciphertext (Hex, truncated)                    |
|--------------------|------------------------------------------------------------|
| Hello, World!      | 9f3a1c72b4e8d05f2a6c3b91e7f4d08a2c5b9e3f1d7a4c6e82b0f5d... |
| Transfer approved. | a7d4e92b1f6c3084d5a2e71c9b4f306d8e3a5c7f0b2d9e14c8a3f6...  |
| Attack at dawn.    | 3b8f2e91c5a4d70e6f1b3c9a2d4e8076f3c9b1a5d7e2f04c6b8a9...   |
| [Audio 4096 bytes] | 3c8f1a5d9e2b7046c4a3f1e82d6b9054a7c1e3f5d8b2a0c9e7f4d...   |

### B. Encryption Algorithm Comparison

Table II compares the cryptographic algorithms employed in the system against representative alternatives across the dimensions of type, key size, security level, computational speed, and primary use case. AES-256 is selected for data encryption due to its combination of Very High security and Very Fast throughput. RSA-2048 is employed for key exchange despite its slower speed because asymmetric key exchange is performed only once per session. SHA-256 provides Very High integrity assurance with Fast computation suitable for per-message hashing.

**TABLE II. COMPARISON OF CRYPTOGRAPHIC ALGORITHMS EVALUATED FOR SYSTEM INTEGRATION**

| Algorithm | Type      | Key Size | Security  | Speed     | Use Case        |
|-----------|-----------|----------|-----------|-----------|-----------------|
| AES-256   | Symmetric | 256-bit  | Very High | Very Fast | Data Encryption |

|          |            |          |           |           |                   |
|----------|------------|----------|-----------|-----------|-------------------|
| RSA-2048 | Asymmetric | 2048-bit | High      | Slow      | Key Exchange      |
| SHA-256  | Hash       | 256-bit  | Very High | Fast      | Integrity Check   |
| DES      | Symmetric  | 56-bit   | Low       | Fast      | Legacy Systems    |
| 3DES     | Symmetric  | 168-bit  | Moderate  | Moderate  | Legacy Systems    |
| ECC-256  | Asymmetric | 256-bit  | Very High | Fast      | Key Exchange      |
| ChaCha20 | Symmetric  | 256-bit  | Very High | Very Fast | Stream Encryption |

### C. Performance Metrics

Table III presents measured encryption, decryption, SHA-256 hashing, RSA key encapsulation, and total end-to-end latency across five payload size categories. Results are reported as means over 1,000 trials. AES-256-CBC encryption and decryption scale linearly with payload size as expected from the  $O(n)$  complexity of CBC mode. SHA-256 computation constitutes a minor fraction of total latency at all payload sizes. RSA encapsulation latency is fixed at approximately 12–15 ms regardless of payload size, as it encrypts only the 256-bit AES key, and is amortized over the session.

**TABLE III. MEASURED CRYPTOGRAPHIC OPERATION LATENCIES BY PAYLOAD SIZE (MEAN OF 1,000 TRIALS)**

| Payload Size | AES Enc. | AES Dec. | SHA-256 | RSA Enc. | Total Latency |
|--------------|----------|----------|---------|----------|---------------|
| 1 KB         | 0.8 ms   | 0.7 ms   | 0.3 ms  | 12.4 ms  | 1.2 ms        |
| 10 KB        | 1.2 ms   | 1.1 ms   | 0.4 ms  | 14.1 ms  | 1.8 ms        |
| 100 KB       | 4.5 ms   | 4.1 ms   | 0.9 ms  | 18.6 ms  | 5.6 ms        |
| 1 MB         | 38.2 ms  | 35.7 ms  | 5.1 ms  | 52.3 ms  | 44.1 ms       |
| 10 MB        | 372 ms   | 349 ms   | 48 ms   | 431 ms   | 428 ms        |

[Figure 2: Line Chart — AES-256 Encryption Time vs. Payload Size (log-log scale)]

**Fig. 2. AES-256-CBC encryption latency as a function of payload size on a log-log scale. The slope of approximately 1.0 confirms linear scaling consistent with  $O(n)$  CBC mode complexity. Error bars represent one standard deviation.**

[Figure 3: Bar Chart — End-to-End Latency Comparison: Normal Transmission vs. Active MITM]

**Fig. 3. Comparison of end-to-end transmission latency under normal operation versus active MITM interception. The additional decrypt-modify-re-encrypt cycle introduces 15–20% overhead for payloads exceeding 100 KB.**

### D. Passive Attack Results

Under passive attack conditions, the Attacker machine successfully logged 100% of TCP payloads. Without the AES session key, all text ciphertext was computationally indecipherable. Audio payloads played through a speaker without decryption produced uniform broadband noise with no identifiable speech or tonal content, confirming complete perceptual unintelligibility. The offline GUI decryption tool successfully recovered all intercepted text messages and audio buffers when supplied with the correct AES key, validating the correctness of the encryption implementation.

### E. Active Attack Results

Under active attack conditions, the Attacker successfully intercepted, decrypted, modified, and re-forwarded 100% of text messages without detection by the Receiver. A string substitution rule replaced the word "approved" with "denied" in all intercepted text messages. The Receiver displayed the modified text without raising an integrity alert, because the Attacker recomputed SHA-256 over the modified plaintext before re-encrypting. For audio modification, a gain scaling factor of 0.1 was applied to all sample amplitudes; the Receiver played back the attenuated audio without alert. This result demonstrates conclusively that SHA-256 hashing without a secret key provides no authentication guarantee against an adversary possessing the encryption key.

### ***F. Integrity Check Accuracy***

Two integrity check scenarios were evaluated. In the first, the passive attacker forwarded all messages unmodified: 100% of messages passed the integrity check (zero false positives). In the second, an artificial bit-flip corruption was introduced by the active attacker without recomputing the hash, simulating an unsophisticated attacker: 100% of corrupted messages were detected by the integrity check (zero false negatives, sensitivity = 1.0). This confirms that SHA-256 provides complete detection of unintentional corruption and unsophisticated tampering, while remaining defeatable by a sophisticated attacker who controls the encryption key.

---

## **VII. SECURITY ANALYSIS**

The implemented system provides strong security guarantees under the passive threat model: an adversary who can only observe ciphertext cannot recover the plaintext without the AES session key, and cannot recover the session key without the RSA private key. The computational hardness assumptions underlying AES-256 (no known attacks better than brute force for 256-bit keys) and RSA-2048 (integer factorization hardness) ensure that passive attacks are computationally infeasible for a classical adversary.

Under the active threat model, however, the system exhibits a critical vulnerability: an adversary who obtains the AES session key can conduct transparent modifications without detection. This vulnerability arises from the use of an unkeyed hash function (SHA-256) for integrity verification. SHA-256 provides data integrity assurance only against adversaries who cannot recompute the hash, i.e., those without the plaintext. An adversary with the session key can always recompute a valid hash for any modified plaintext. The solution to this vulnerability is the introduction of authentication: either HMAC-SHA-256 with a secret key unknown to the attacker, or an asymmetric digital signature (RSA-PSS or ECDSA) over the message digest.

Additionally, the current system performs no certificate-based mutual authentication of public keys. A fully active MITM attacker could substitute its own RSA public key for the Receiver's during the key exchange phase, intercepting the AES session key in plaintext. This attack is prevented in production systems by binding public keys to verified identities through an X.509 PKI or a web-of-trust model.

---

## **VIII. CONCLUSION**

This paper has presented the complete design, implementation, and experimental evaluation of a hybrid cryptographic secure communication system subjected to both passive and active Man-in-the-Middle attacks. The system integrates AES-256-CBC for data confidentiality, RSA-2048-OAEP for session key exchange, and SHA-256 for data integrity, deployed across three physical machines in a realistic LAN environment supporting both text and real-time audio communication modalities.

Experimental results confirm that the hybrid encryption pipeline provides computationally intractable resistance against passive eavesdropping for both text and audio content. The AES-256 implementation achieves sub-6 ms total latency for 100 KB payloads, demonstrating practical suitability for real-time secure communication. The integrity verification mechanism successfully detects 100% of unauthenticated modification attempts, validating the correctness of the SHA-256 pipeline.

The active attack simulation reveals the fundamental security limitation of the system: without mutual authentication, an adversary possessing the session key can modify content transparently, defeating the SHA-256 integrity check by recomputing the hash over tampered plaintext. This result provides a concrete, empirically grounded demonstration of the cryptographic principle that confidentiality and integrity alone are insufficient without authentication, a principle that motivates the design of authenticated encryption schemes and certificate-based mutual authentication in production-grade security protocols such as TLS 1.3.



## IX. FUTURE WORK

Several extensions to the current system are identified for future development:

- **Digital Signature Integration:** Adding RSA-PSS or ECDSA digital signatures over message digests would provide sender authentication, preventing transparent active modification even when the session key is compromised, as the attacker cannot forge a valid signature without the sender's private key.
- **Certificate-Based Mutual Authentication:** Deploying an X.509 PKI would bind RSA public keys to verified identities, closing the authentication gap that enables a fully active MITM attacker to substitute its own public key during the handshake phase.
- **TLS 1.3 Migration:** Replacing the custom handshake protocol with standards-compliant TLS 1.3 would provide ephemeral Diffie-Hellman key exchange (ECDHE), forward secrecy, authenticated encryption (AES-GCM or ChaCha20-Poly1305), and resistance to downgrade attacks, yielding a production-grade security posture.
- **Authenticated Encryption (AEAD):** Replacing the separate AES-CBC + SHA-256 pipeline with AES-256-GCM would provide combined confidentiality and authenticated integrity in a single, more efficient cryptographic operation, eliminating the authentication gap identified in this work.
- **Real-Time Audio Performance Optimization:** Leveraging AES-NI hardware acceleration instructions and parallelizing per-frame encryption across CPU cores would reduce per-frame encryption latency to sub-millisecond levels, enabling high-quality real-time audio communication over encrypted channels.
- **Perfect Forward Secrecy:** Replacing static RSA key encapsulation with ephemeral ECDHE key agreement would ensure that compromise of the long-term RSA private key does not expose historical session traffic.
- **Wireless and Mobile Deployment:** Extending the system to Wi-Fi and LTE/5G environments would introduce additional attack surfaces including rogue access points, IMSI catchers, and RF-layer interception, warranting further security analysis and implementation of 802.1X or similar wireless authentication mechanisms.

---

## References.

- [1] W. Stallings, *Network Security Essentials: Applications and Standards*, 6th ed. Hoboken, NJ: Pearson Education, 2017.
- [2] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, NIST, Gaithersburg, MD, Nov. 2001.
- [3] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978.
- [4] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, NIST, Gaithersburg, MD, Aug. 2015.
- [5] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018.
- [6] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd ed. New York, NY: Wiley, 1996.
- [7] J. Daemen and V. Rijmen, *The Design of Rijndael: AES — The Advanced Encryption Standard*. Berlin, Germany: Springer, 2002.
- [8] F. Callegati, W. Cerroni, and M. Ramilli, "Man-in-the-Middle Attack to the HTTPS Protocol," *IEEE Security Privacy*, vol. 7, no. 1, pp. 78–81, Jan./Feb. 2009.
- [9] M. Conti, N. Dragoni, and V. Lesyk, "A Survey of Man-In-The-Middle Attacks," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 3, pp. 2027–2051, 3rd Quart. 2016.
- [10] B. A. Forouzan, *Data Communications and Networking*, 5th ed. New York, NY: McGraw-Hill, 2013.
- [11] Python Software Foundation, "cryptography — Cryptographic Recipes and Primitives for Python," PyPI, 2023. [Online]. Available: <https://pypi.org/project/cryptography/>
- [12] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1997.